

Student Performance Prediction

Using Machine Learning

A small Flask web app that predicts whether a student will **pass or fail** from three inputs: previous exam score, hours studied and attendance. It trains two scikit-learn classifiers (Logistic Regression and Decision Tree) and shows both predictions and both accuracies on the page.

Final-year academic project, B.Sc. Artificial Intelligence & Machine Learning, Suguna College of Arts and Science (Bharathiar University), 2026.

What it does

- Loads `dataset.csv` , or creates a 20-row sample dataset if the file does not exist.
- Splits the data 80/20 (`random_state=42`) and trains a Logistic Regression model and a Decision Tree model.
- Evaluates both models on the test split with accuracy and a confusion matrix (printed in the terminal).
- Saves the trained models as `logistic_model.pkl` and `decision_tree_model.pkl` , and reuses them on later runs.
- Serves a web form. You enter the three values and the page shows the Pass/Fail prediction from each model, along with each model's test accuracy.

Tech stack

Area	Tools
Language	Python
ML	<code>scikit-learn (LogisticRegression , DecisionTreeClassifier , train_test_split , accuracy_score , confusion_matrix)</code>
Data	pandas
Web	Flask, HTML, CSS
Model storage	pickle

Dataset

Column	Description	Type
<code>previous_score</code>	Marks in the previous exam (out of 100)	Numeric
<code>hours_studied</code>	Hours studied for the current exam	Numeric
<code>attendance</code>	Percentage of classes attended	Numeric

Column	Description	Type
pass	Target: 1 = Pass, 0 = Fail	Binary

The built-in sample has **20 rows** (12 pass, 8 fail). It is a small hand-written sample, not real student data. To use your own data, put a `dataset.csv` with the same four columns next to `app.py` and delete the two `.pkl` files so the models retrain.

Results (with the included sample data)

Model	Test accuracy	Confusion matrix (rows = actual, columns = predicted)
Logistic Regression	100%	[[1, 0], [0, 3]]
Decision Tree	100%	[[1, 0], [0, 3]]

Read these numbers carefully. With 20 rows and an 80/20 split, the test set has only 4 students. Both models get all 4 right, so this run cannot show that one model is better than the other, and 100% here does not mean the model would do well on real data. Treat this project as a working end-to-end pipeline (data, training, evaluation, web deployment), not as a validated predictor.

Project structure

```
student-performance-prediction/  
├─ app.py          # data loading,  
training, evaluation, Flask routes  
├─ templates/  
│   └─ index.html  # input form and  
results page  
├─ dataset.csv      # created  
automatically on first run if missing  
├─ logistic_model.pkl # created  
automatically after first run  
├─ decision_tree_model.pkl  
└─ README.md
```

Example

Entering 78 / 5 / 90 (previous score / hours studied / attendance) returns **Pass** from both models. Entering 55 / 2 / 60 returns **Fail** from both. These values are rows from the sample dataset, so they show that the app runs correctly, not that it generalises.

Limitations

- Very small sample dataset, so the accuracy figures are not reliable.
- Only 4 test rows, and no cross-validation.
- No feature scaling or missing-value handling in the code.
- Input errors show a generic "Invalid Input" message.
- Models are loaded from the saved .pkl files, so changing the dataset requires deleting them to retrain.

- The app runs with Flask's development server (`debug=True`), which is not meant for production.

Possible improvements

- Train on a larger real dataset with more features.
- Compare models with k-fold cross-validation and report precision and recall.
- Add feature scaling and input validation.
- Show the confusion matrix and feature importance in the web page.

Author

Samasri M

B.Sc. Artificial Intelligence & Machine Learning (2026)

[LinkedIn](#) · samasrimanikandan@gmail.com

<!-- Optional: add a screenshot of the running app at docs/screenshot.png, then add: ![App screenshot] (docs/screenshot.png) -->